UNIVERSIDAD DEL VALLE

750027C - Desarrollo de Software III

Guía académica

Dojo: Notificaciones asíncronas con RabbitMQ

Guía paso a paso

FastAPI · Docker · RabbitMQ

Presentado por	**PhD. Manuel Alejandro Pastrana Pardo** Profesor Nombrado — EISC, Facultad de Ingeniería Email: manuel.pastrana@correounivalle.edu.co
Tema	Microservicios con comunicación asíncrona.
Lugar y año	Cali, Colombia — 2026

Tabla de Contenido

Contenido

Tabla de Contenido 2
Objetivo del Laboratorio 3
Prerrequisitos 3
Contexto arquitectónico 3
Paso 1: Levantar RabbitMQ con Docker 4
Paso 2: Crear el proyecto notificador-api 7
Paso 3: Definir el Esquema del Evento (el "Contrato") 10
Paso 4: Implementar el Productor con aio-pika 11
Paso 5: Exponer el Endpoint POST /notificaciones 14
Paso 6: Probar el Productor (Swagger + consola de RabbitMQ) 15
Paso 7: Crear el Proyecto notificador-worker 18
Paso 8: Implementar el Consumidor con ack explícito 19
Paso 9: Persistir lo Consumido (SQLite) 22
Paso 10: Dockerizar Ambos Servicios 26
- notificador-api 26
- notificador-worker 27

Paso 11: docker-compose.yml Completo (RabbitMQ + API + Worker) 28
- Deteniendo los procesos sueltos 28
- docker-compose.yml final, en la raíz del proyecto 28

Paso 12: Prueba End-to-End desde cero 30
Solución de problemas comunes 32
- Comandos partidos en varias líneas 32
- Git Bash en Windows convierte rutas absolutas de Linux 32
- localhost contra rabbitmq en RABBITMQ_URL 32
- ACCESS_REFUSED al conectar 33
- PRECONDITION_FAILED: inequivalent arg 33
- Publiqué un mensaje, la API respondió 200, pero no llegó a ninguna cola 33
- Mensajes en estado Unacked que no se mueven 33
- Las colas desaparecen después de recrear el contenedor 33
- Puertos 5672 o 15672 ocupados 33
- VS Code crea archivos en la carpeta seleccionada 33
- KeyboardInterrupt al detener el worker con Ctrl+C 33

Actividad de reflexión 34
Posibles respuestas 34

Objetivo del Laboratorio

Al finalizar este laboratorio, el estudiante será capaz de conectar una API REST a RabbitMQ, publicar mensajes desde un servicio y consumirlos de forma asíncrona desde otro servicio completamente independiente, confirmando cada mensaje procesado y persistiéndolo de manera duradera.

El énfasis no está en dominar RabbitMQ a fondo, sino en entender cómo funciona una cola de mensajes y en qué se diferencia de un log de eventos como Kafka: dos servicios que nunca se llaman directamente, comunicados solo a través de un exchange y una cola, cada uno desplegable y escalable de forma independiente.

Este laboratorio corre completamente en local, usando contenedores Docker. No incluye despliegue en la nube.

Prerrequisitos

- Docker Desktop instalado y corriendo, con soporte para Docker Compose v2.
- Python 3.10 o superior instalado.
- Visual Studio Code (o un editor equivalente) con una terminal integrada. En Windows se recomienda Git Bash.
- Conocimientos básicos de Python y de la línea de comandos.
- No se requiere experiencia previa con RabbitMQ ni con Kafka.

Contexto arquitectónico

El laboratorio está compuesto por dos servicios independientes y un broker de mensajería:

- **notificador-api**: una API construida con FastAPI que expone un endpoint para crear notificaciones. Al recibir una petición, publica un mensaje en RabbitMQ y responde de inmediato, sin esperar a que nadie lo consuma.
- **notificador-worker**: un proceso independiente, sin interfaz HTTP, que toma los mensajes de una cola, los procesa, confirma cada uno (ack) y los persiste en una base de datos SQLite.
- **RabbitMQ**: el broker que conecta a los dos anteriores. Ninguno conoce la dirección del otro; ambos solo conocen el nombre del exchange, de la cola y de la routing key.

RabbitMQ introduce cinco conceptos que se usan a lo largo de toda la guía:

Concepto	Qué es	En este laboratorio
Exchange	Recibe los mensajes del productor y decide a qué cola(s) enviarlos	notificaciones.exchange (tipo direct)
Routing key	Etiqueta que el productor pone al mensaje y que el exchange usa para enrutarlo	notificacion.creada
Queue	Buffer ordenado donde esperan los mensajes hasta que un consumidor los procesa	notificaciones.worker (durable)
Binding	Regla que une un exchange con una cola para una routing key	notificaciones.exchange con notificaciones.worker mediante notificacion.creada
Ack	Confirmación del consumidor; solo entonces RabbitMQ borra el mensaje de la cola	mensaje.ack() después de guardar en SQLite

Dos diferencias con Kafka condicionan el diseño del laboratorio. Primera: el productor nunca escribe directamente en una cola, sino en un exchange. Segunda: un mensaje que llega a un exchange sin ninguna cola enlazada se descarta en silencio, mientras que en Kafka el tópico lo conserva. Por eso la cola la declara el consumidor, y en el Paso 6 se crea a mano para observar este comportamiento.

La separación entre API y worker es deliberada. En un sistema real podrían vivir en repositorios distintos, desplegarse en momentos distintos y escalar de forma independiente. Lo único que los mantiene compatibles es que ambos respeten el mismo contrato de datos (definido con Pydantic en schemas.py) y los mismos nombres de exchange y routing key, no que compartan código. Cada servicio tiene su propia copia del esquema.

El productor y el worker corren en procesos separados desde el comienzo del laboratorio, no solo al final. Es intencional: mezclarlos en un mismo proceso eliminaría la razón principal para usar un broker.

Paso 1: Levantar RabbitMQ con Docker

RabbitMQ corre en un único contenedor. Se usa la imagen oficial rabbitmq con la etiqueta management, que incluye la consola de administración web: una interfaz en el navegador para ver exchanges, colas y mensajes. Esa consola será la principal herramienta de verificación del laboratorio.

1. Crea la carpeta raíz del proyecto y entra en ella:

```
mkdir dojo-rabbitmq-notificaciones
cd dojo-rabbitmq-notificaciones
```

2. Abre esa carpeta en VS Code (File > Open Folder).
3. Crea un archivo docker-compose.yml en la raíz con este contenido:

```
# Imagen oficial de RabbitMQ con el plugin de administración web.
services:
  rabbitmq:
    image: rabbitmq:4-management
```

```
    container_name: rabbitmq
    # Nombre de host fijo. RabbitMQ identifica su nodo como
    # "rabbit@<hostname>" y guarda sus datos bajo ese nombre. Con el
    # hostname por defecto de Docker (el id del contenedor), cada
    # contenedor nuevo sería un nodo distinto y no vería las colas
    # guardadas en el volumen.
    hostname: rabbitmq
    ports:
      # 5672: protocolo AMQP, el que usan los clientes (API y worker).
      - '5672:5672'
      # 15672: consola de administración web.
      - '15672:15672'
    environment:
      # Usuario y clave propios para el laboratorio, en lugar del
      # usuario guest/guest que trae RabbitMQ por defecto (restringido a
      # conexiones locales en una instalación estándar). Solo se aplican
      # la primera vez que se crea el volumen de datos.
      RABBITMQ_DEFAULT_USER: notificador
      RABBITMQ_DEFAULT_PASS: notificador123
    volumes:
      # Las colas y los mensajes persistentes sobreviven a un
      # docker-compose down/up.
      - rabbitmq_data:/var/lib/rabbitmq
    # Evita que otros servicios intenten conectarse antes de que
    # RabbitMQ esté listo para aceptar conexiones.
    healthcheck:
      test: ['CMD', 'rabbitmq-diagnostics', '-q', 'ping']
      interval: 10s
      timeout: 10s
      retries: 5
      start_period: 20s

volumes:
  rabbitmq_data:
```

4. Guarda el archivo y levanta el contenedor:

```
docker-compose up -d
```

```
Manuel Pastrana@DESKTOP-7FO66S7 MINGW64 /f/Workspace/dojo-rabbitmq-notificaciones
$ docker-compose up -d
[+] Running 13/13
 ✓ rabbitmq Pulled
[+] Running 3/3
 ✓ Network dojo-rabbitmq-notificaciones_default       Created
 ✓ Volume dojo-rabbitmq-notificaciones_rabbitmq_data  Created
 ✓ Container rabbitmq                                 Started
```

```
docker ps
```

```
Manuel Pastrana@DESKTOP-7FO66S7 MINGW64 /f/Workspace/dojo-rabbitmq-notificaciones
$ docker-compose ps -a
NAME       IMAGE                  COMMAND                  SERVICE    CREATED         STATUS                   PORTS
rabbitmq   rabbitmq:4-management  "docker-entrypoint.s…"   rabbitmq   3 minutes ago   Up 3 minutes (healthy)   0.0.0.0:5672->5672/t
:]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
```

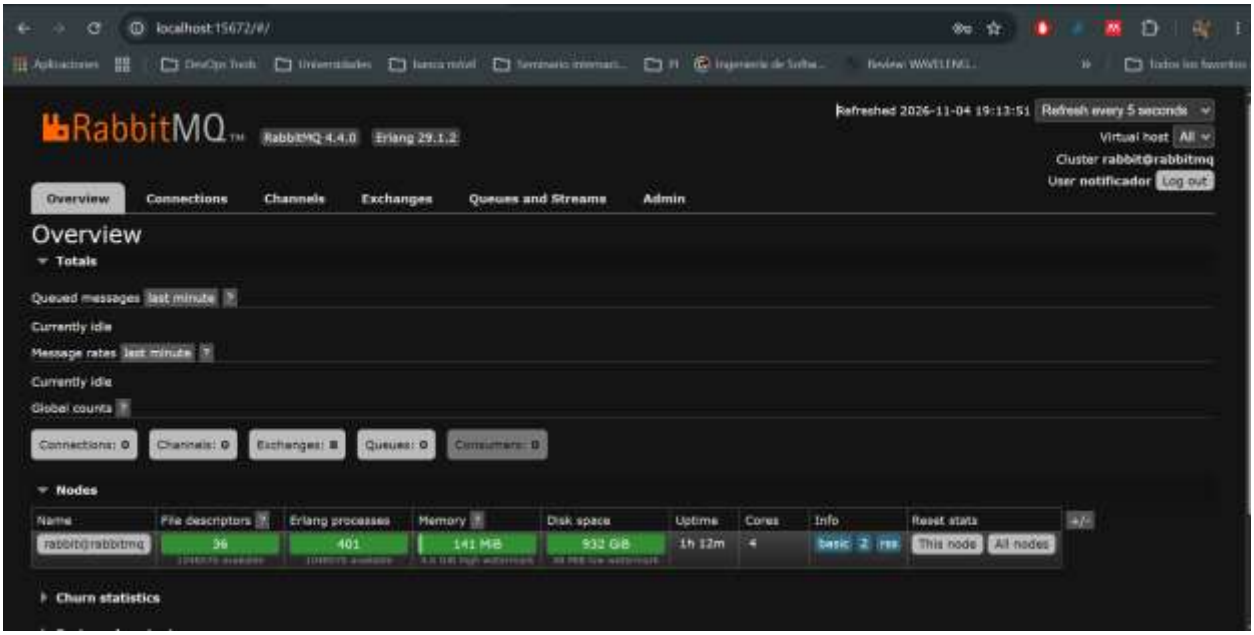
Debe aparecer un contenedor llamado rabbitmq con estado Up. Tras 20 a 30 segundos el estado cambia a healthy.

5. Checkpoint de validación: confirma que RabbitMQ responde listando las colas (estará vacío, pero si el comando responde sin error, el broker está sano):

```
docker exec rabbitmq rabbitmqctl list_queues
```

```
Manuel Pastrana@DESKTOP-7FO66S7 MINGW64 /f/Workspace/dojo-rabbitmq-notificaciones
$ docker exec rabbitmq rabbitmqctl list_users
Listing users ...
user    tags
notificador     [administrator]
```

6. Abre la consola de administración en http://localhost:15672 e ingresa con el usuario notificador y la clave notificador123, escribiéndolos a mano y sin copiarlos ni pegarlos (al pegarlos RabbitMQ puede rechazarlos con el error Not_Authorized). Recorre las pestañas Overview, Connections, Channels, Exchanges y Queues and Streams, que se describen a continuación. Por ahora no hay conexiones ni colas, y en Exchanges solo aparecen los exchanges que RabbitMQ crea por defecto.



> **Nota para Windows (Git Bash):** a diferencia de la guía de Kafka, los comandos de este paso no necesitan el prefijo MSYS_NO_PATHCONV=1, porque ningún argumento empieza con /. Ese prefijo solo se usa más adelante, al consultar SQLite en la ruta /app/data dentro del contenedor.
>
> **Nota sobre las credenciales:** notificador123 es una clave de laboratorio, visible en texto plano en el docker-compose.yml. En un despliegue real las credenciales se entregan mediante un gestor de secretos.
>
> **Definiciones**:
>
> **Overview.** Es el resumen general del broker. Muestra los totales de conexiones, canales, exchanges, colas y consumidores, las gráficas de mensajes publicados y entregados por segundo, el nombre del nodo (rabbit@rabbitmq) y los puertos en los que escucha (5672 para

AMQP y 15672 para la consola web). Sirve para confirmar de un vistazo que el broker está sano y para ver el flujo de mensajes mientras se hacen las pruebas.

Connections. Lista las conexiones TCP que los clientes han abierto hacia el broker, con el usuario, el origen y el estado de cada una. Hay una conexión por proceso cliente. En este laboratorio sirve para comprobar que la API (Paso 4) y el worker (Paso 8) lograron conectarse.

Channels. Un canal es una línea virtual dentro de una conexión, y es por ahí donde realmente se publica y se consume. Una conexión es el cable entre el cliente y el broker, y los canales son las líneas que viajan dentro de ese cable. Esta pestaña muestra el prefetch de cada canal y cuántos mensajes tiene entregados sin confirmar. Sirve para verificar el prefetch_count=1 del worker y detectar mensajes a los que les falta el ack.

Exchanges. Lista los exchanges existentes, con su tipo (direct, fanout, topic o headers) y si son durables. El exchange recibe los mensajes del productor y decide a qué cola enviarlos. Al inicio solo aparecen los que RabbitMQ trae por defecto (el exchange sin nombre y los que empiezan con amq.). Después de levantar la API aparecerá notificaciones.exchange.

Queues and Streams. Lista las colas, con su tipo (Classic, Quorum o Stream), si son durables y cuántos mensajes tienen: Ready (esperando a un consumidor), Unacked (entregados pero sin confirmar) y Total. Desde aquí también se crean colas, se agregan bindings y se leen mensajes con Get messages. En el Paso 6 se usa para crear la cola notificaciones.worker y enlazarla al exchange.

Por eso Connections y Channels están vacías en este paso: todavía no hay ningún cliente conectado.

Paso 2: Crear el proyecto notificador-api

1. Desde la raíz del proyecto, crea la carpeta del servicio y entra en ella:

```
mkdir notificador-api
cd notificador-api
```

2. Crea y activa un entorno virtual de Python. En Git Bash sobre Windows, la ruta de activación usa Scripts, no bin:

Primero ejecuta: `python -m venv venv`



y luego `source venv/Scripts/actívate`



El símbolo (venv) debe aparecer al inicio del prompt de la terminal.

Nota: al crear un archivo nuevo en VS Code (clic derecho > New File), el archivo se crea dentro de la carpeta que esté seleccionada en el explorador en ese momento. Verifica siempre el

> breadcrumb sobre el editor antes de pegar contenido; es fácil terminar creando un archivo dentro de venv/ por accidente.

3. Con notificador-api seleccionada (no venv), crea requirements.txt:

```
fastapi
uvicorn[standard]
aio-pika
pydantic
```

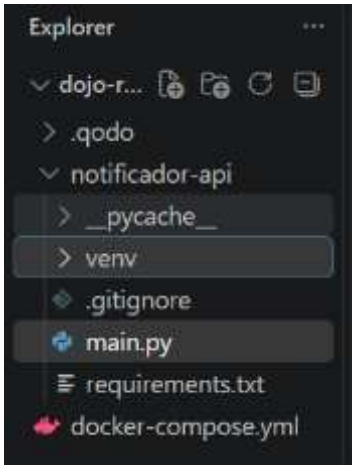
aio-pika es el cliente de RabbitMQ para asyncio. Es el equivalente de aiokafka en la guía de Kafka: permite publicar y consumir con async/await sin bloquear el event loop de FastAPI.

4. Instala las dependencias:

```
pip install -r requirements.txt
```

```
(venv)
Manuel Pastrana@DESKTOP-7FO66S7 MINGW64 /f/Workspace/dojo-rabbitmq-notificaciones/notificador-api
$ ls
requirements.txt  venv/
(venv)
Manuel Pastrana@DESKTOP-7FO66S7 MINGW64 /f/Workspace/dojo-rabbitmq-notificaciones/notificador-api
$ pip install -r requirements.txt
Collecting fastapi (from -r requirements.txt (line 1))
  Downloading fastapi-0.162.0-py3-none-any.whl.metadata (26 kB)
Collecting aio-pika (from -r requirements.txt (line 3))
  Downloading aio_pika-10.2.2-py3-none-any.whl.metadata (16 kB)
Collecting pydantic (from -r requirements.txt (line 4))
  Downloading pydantic-2.14.2-py3-none-any.whl.metadata (137 kB)
Collecting uvicorn[standard] (from -r requirements.txt (line 2))
  Downloading uvicorn-0.54.0-py3-none-any.whl.metadata (6.6 kB)
Collecting starlette>=0.46.0 (from fastapi->-r requirements.txt (line 1))
```

5. Crea main.py, al mismo nivel que requirements.txt, con un endpoint mínimo, solo para confirmar que el servidor levanta antes de agregar la lógica de mensajería.



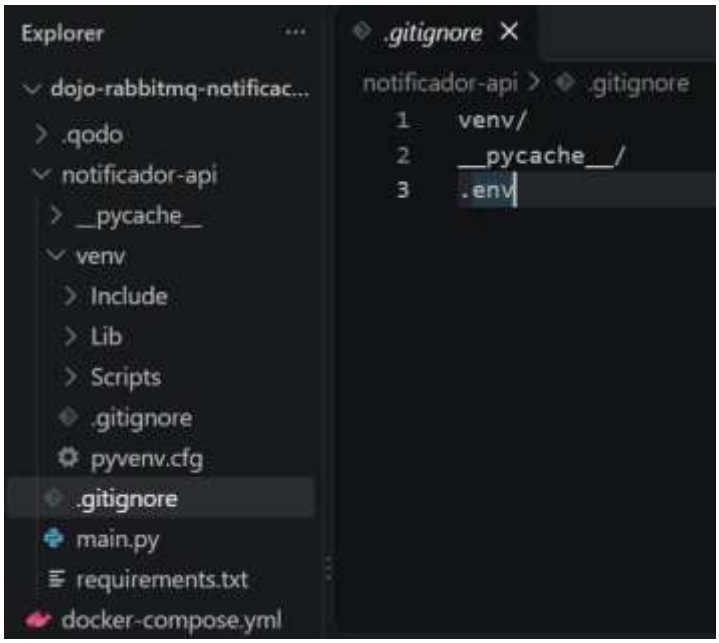
6. Intenta crear primero tu solo el archivo. Luego compara con el siguiente código de guía:

```
from fastapi import FastAPI

app = FastAPI(title='Notificador API')
```

```
@app.get('/health')
def health():
    return {'status': 'UP', 'service': 'notificador-api'}
```

7. Crea .gitignore en la carpeta notificador-api, al mismo nivel que requirements.txt y main.py (no dentro de venv/). Este archivo le indica a git qué no debe versionar ni el entorno virtual, ni el caché de Python (__pycache__/) y tampoco el archivo .env, donde suelen guardarse credenciales.

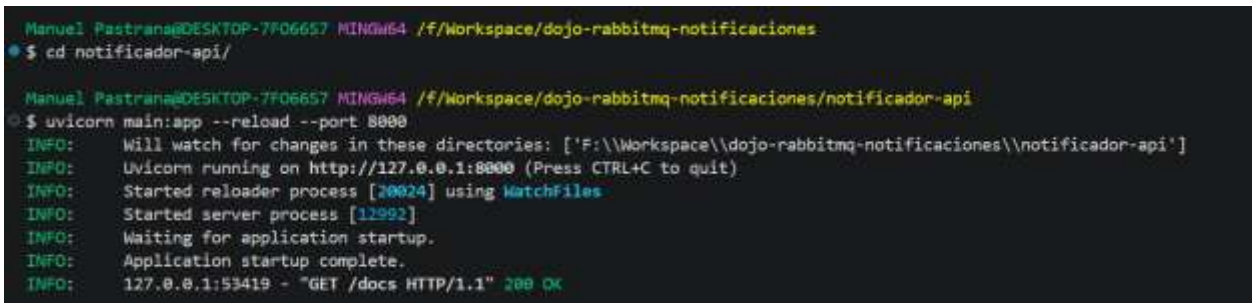


El contenido del archivo es el siguiente

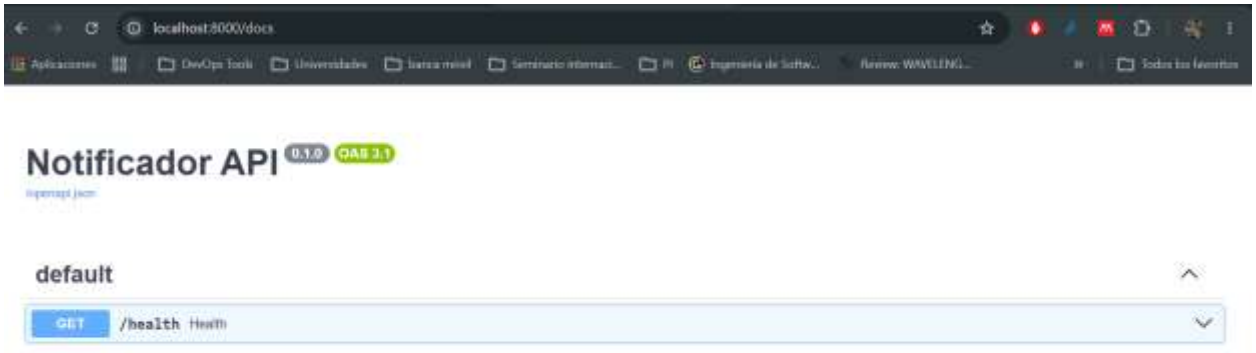
```
venv/
__pycache__/
.env
```

> **Nota:** dentro de venv/ verás otro .gitignore. Lo genera Python 3.13 automáticamente al crear el entorno y no hay que tocarlo. El que se pide en este paso es un segundo archivo, el de la carpeta notificador-api.

8. Levanta el servidor. Recuerda que para poder ejecutar el comando **uvicorn main:app --reload --port 8000** se debe estar dentro de la carpeta del notificador



9. Abre http://localhost:8000/docs en el navegador y prueba el endpoint /health desde Swagger UI. Debe responder {"status": "UP", "service": "notificador-api"}.



Paso 3: Definir el Esquema del Evento (el "Contrato")

Esta clase materializa el desacoplamiento entre los dos servicios: notificador-api y notificador-worker tendrán cada uno su propia copia, sin importarla del otro, porque en el mundo real son repositorios y despliegues independientes. **Lo que los mantiene compatibles es que ambos respeten el mismo contrato, no que compartan código.**

1. Con notificador-api seleccionada, crea schemas.py:

```
from datetime import datetime, timezone
from uuid import uuid4

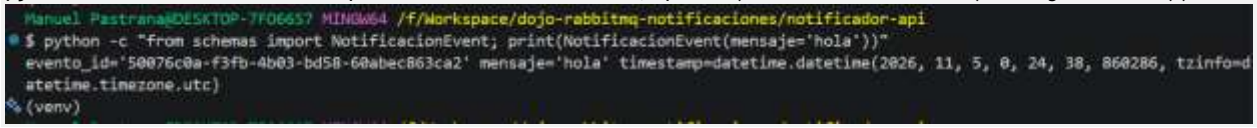
from pydantic import BaseModel, Field



# evento_id y timestamp tienen default_factory: se generan solos al
# construir el objeto, sin que quien llame a la API tenga que enviarlos.
# El cliente solo necesita enviar 'mensaje'.
#
# El timestamp usa datetime.now(timezone.utc) y no datetime.utcnow(),
# que está deprecado desde Python 3.12 y devuelve una fecha sin zona
# horaria.
class NotificacionEvent(BaseModel):
    evento_id: str = Field(default_factory=lambda: str(uuid4()))
    mensaje: str
    timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
```

2. Valida que el modelo funciona, sin necesidad de levantar el servidor:

```
python -c "from schemas import NotificacionEvent; print(NotificacionEvent(mensaje='hola'))"
```



Debe imprimir algo como: evento_id='...' mensaje='hola' timestamp=datetime.datetime(2026, 10, 29, 16, 0, 0, 123456, tzinfo=datetime.timezone.utc)

Si aparece ModuleNotFoundError: No module named 'schemas', la terminal no está en la carpeta notificador-api o el archivo quedó en otra carpeta (revisa con pwd y ls).

Paso 4: Implementar el Productor con aio-pika

1. Con notificador-api seleccionada, crea **producer.py**, al mismo nivel que main.py y schemas.py. Después de crearlo puedes pegar el siguiente contenido:

```
import json
import os

import aio_pika
from aio_pika import DeliveryMode, ExchangeType, Message
from aio_pika.abc import AbstractExchange, AbstractRobustConnection

# Se lee de variable de entorno con un valor por defecto para desarrollo
# local. Al dockerizar (Paso 10), 'localhost' cambiará a 'rabbitmq' (el
# nombre del servicio dentro de la red de Docker) y este archivo no se
# toca: solo la variable de entorno en el compose.
RABBITMQ_URL = os.environ.get(
    'RABBITMQ_URL', 'amqp://notificador:notificador123@localhost:5672/'
)

# Nombres que el worker también debe conocer. Son parte del contrato, igual
# que el esquema del evento.
EXCHANGE_NAME = 'notificaciones.exchange'
ROUTING_KEY = 'notificacion.creada'

# La conexión y el exchange se crean una sola vez y se reutilizan durante
# toda la vida del proceso. Abrir una conexión por cada request sería
# costoso e innecesario.
connection: AbstractRobustConnection | None = None
exchange: AbstractExchange | None = None


async def start_producer() -> None:
    """Abre la conexión con RabbitMQ y declara el exchange.

    Se llama una sola vez, en el lifespan de FastAPI (ver main.py). Si
    RabbitMQ no está disponible en este punto, connect_robust lanza una
    excepción aquí mismo: este es el checkpoint donde se detecta si el
    contenedor no está corriendo o no es alcanzable.
    """
    global connection, exchange
    # connect_robust reconecta solo si la conexión se cae después de haberse
    # establecido.
    connection = await aio_pika.connect_robust(RABBITMQ_URL)
    # Con la configuración por defecto, el canal usa publisher confirms:
    # publish() espera la confirmación del broker antes de retornar.
    channel = await connection.channel()
```

```
    # Declarar es idempotente: si el exchange ya existe con los mismos
    # parámetros, no pasa nada. durable=True hace que sobreviva a un
    # reinicio del broker.
    exchange = await channel.declare_exchange(
        EXCHANGE_NAME, ExchangeType.DIRECT, durable=True
    )


async def stop_producer() -> None:
    """Cierra la conexión limpiamente al apagar la aplicación."""
    if connection is not None:
        await connection.close()


async def publish_notificacion(evento: dict) -> None:
    """Publica un evento en el exchange de notificaciones."""
    if exchange is None:
        raise RuntimeError('El productor de RabbitMQ no ha sido inicializado.')

    mensaje = Message(
        # RabbitMQ transporta bytes, no objetos Python: dict -> JSON -> bytes.
        body=json.dumps(evento).encode('utf-8'),
        content_type='application/json',
        # PERSISTENT hace que el broker escriba el mensaje en disco. Junto
        # con una cola durable, el mensaje sobrevive a un reinicio de
        # RabbitMQ. Sin esto, el mensaje solo vive en memoria.
        delivery_mode=DeliveryMode.PERSISTENT,
        message_id=evento['evento_id'],
    )
    await exchange.publish(mensaje, routing_key=ROUTING_KEY)
```

2. Conecta el ciclo de vida del productor con el arranque de FastAPI. Reemplaza el contenido de main.py por:

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from producer import start_producer, stop_producer

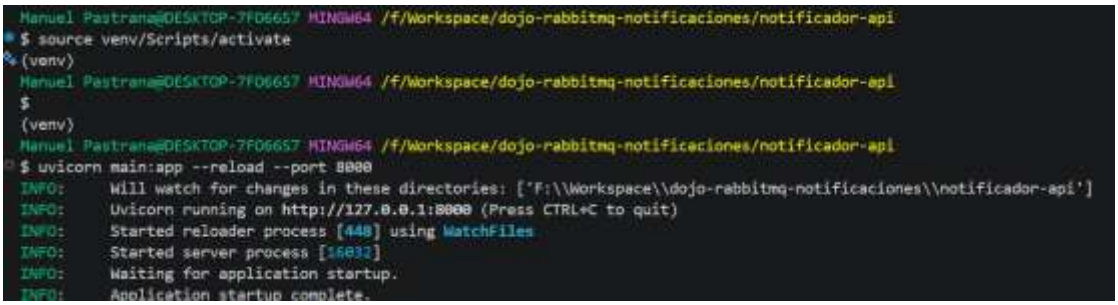
# El lifespan es el código que corre antes de que la app empiece a recibir
# requests y después de que termine de recibirlas. Se usa para abrir y
# cerrar la conexión con RabbitMQ una sola vez.
@asynccontextmanager
async def lifespan(app: FastAPI):
    await start_producer()
    yield
    await stop_producer()

app = FastAPI(title='Notificador API', lifespan=lifespan)

@app.get('/health')
def health():
    return {'status': 'UP', 'service': 'notificador-api'}
```

3. Levanta el servidor de nuevo. Asegúrate primero que estas usando el venv creado. Por si acaso.

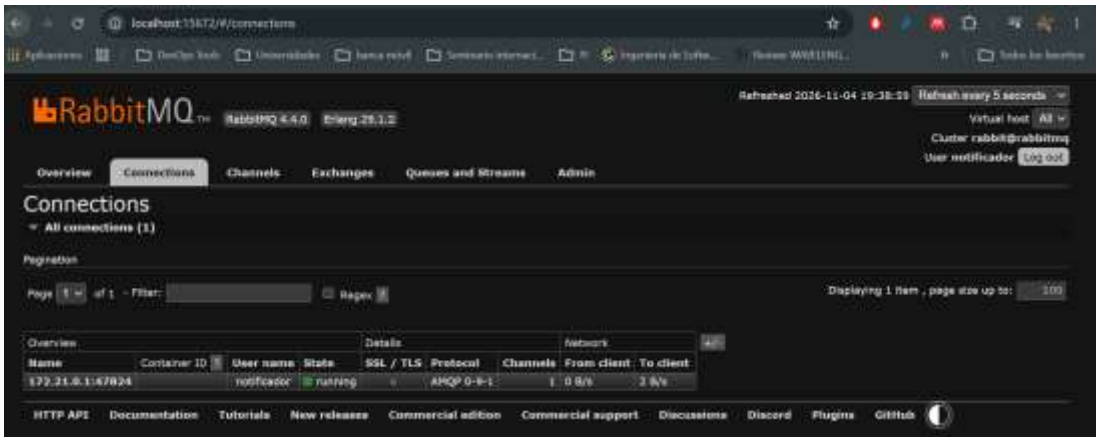
```
source venv/Scripts/activate
uvicorn main:app --reload --port 8000
```



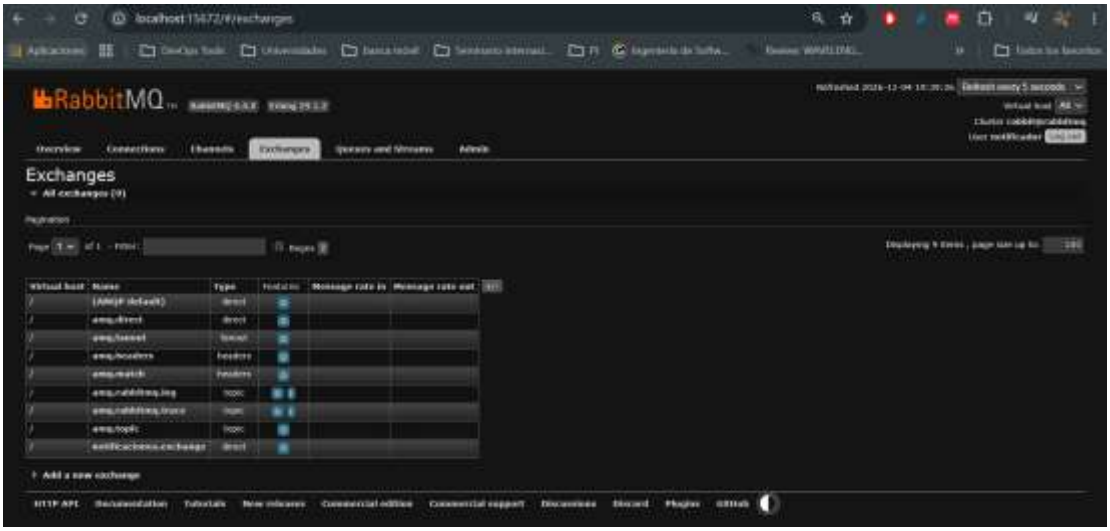
Este es el checkpoint importante: si aio-pika no logra conectarse, falla aquí mismo al arrancar. Si el log muestra Application startup complete, la API ya está hablando con RabbitMQ.

4. Confirma en la consola (http://localhost:15672):

- En **Connections** aparece una conexión abierta desde la API.



- En **Exchanges** aparece **notificaciones.exchange**, de tipo direct y con la marca D (durable).



> **Error común: ConnectionRefusedError o AMQPConnectionError.** RabbitMQ no está corriendo o el puerto 5672 no está publicado. Revisa con docker ps que el contenedor rabbitmq esté Up.
>
> **Error común: ACCESS_REFUSED.** El usuario o la clave de RABBITMQ_URL no coinciden con los del docker-compose.yml. Recuerda que RABBITMQ_DEFAULT_USER y RABBITMQ_DEFAULT_PASS solo se aplican al crear el volumen; si se cambian después, se necesita docker-compose down -v para que surtan efecto.

Paso 5: Exponer el Endpoint POST /notificaciones

1. Reemplaza el contenido completo de main.py por:

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from producer import publish_notificacion, start_producer, stop_producer
from schemas import NotificacionEvent

# El lifespan es el código que corre antes de que la app empiece a recibir
# requests y después de que termine de recibirlas. Se usa para abrir y
# cerrar la conexión con RabbitMQ una sola vez.
@asynccontextmanager
async def lifespan(app: FastAPI):
    await start_producer()
    yield
    await stop_producer()

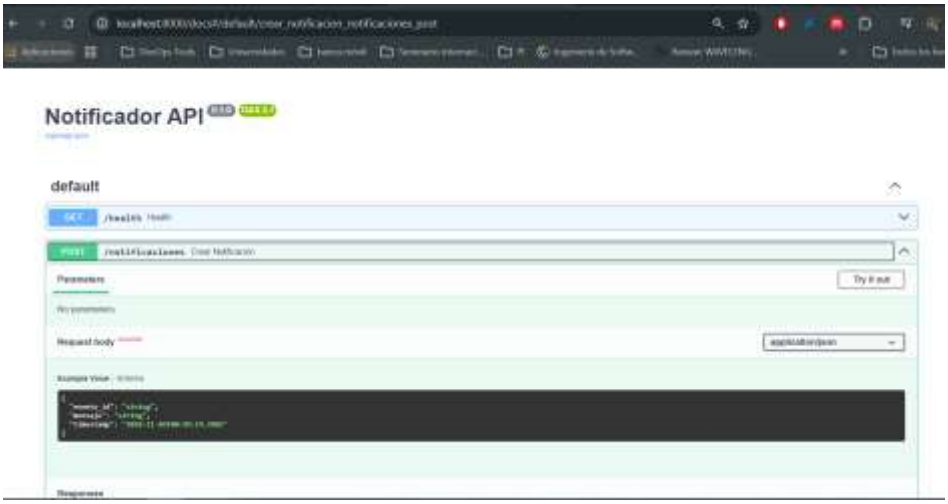
app = FastAPI(title='Notificador API', lifespan=lifespan)

@app.get('/health')
def health():
    return {'status': 'UP', 'service': 'notificador-api'}

# response_model le dice a FastAPI qué forma tiene la respuesta, y por eso
# Swagger sabe mostrarla. Como el parámetro 'evento' está tipado con
# NotificacionEvent, FastAPI valida el body contra ese esquema antes de
# ejecutar el código de la función.
@app.post('/notificaciones', response_model=NotificacionEvent)
async def crear_notificacion(evento: NotificacionEvent) -> NotificacionEvent:
    # El cliente solo necesita enviar {'mensaje': '...'}. Como evento_id y
    # timestamp tienen default_factory, Pydantic los genera al construir
    # el objeto, aunque el cliente nunca los haya enviado.
    #
    # model_dump(mode='json') convierte el objeto a un diccionario donde el
    # timestamp (un datetime de Python) queda como string ISO 8601. Sin
    # mode='json', json.dumps (en producer.py) fallaría, porque no sabe
    # convertir un datetime por sí solo.
    await publish_notificacion(evento.model_dump(mode='json'))

    # Se devuelve el mismo evento publicado, para que quien llamó al
    # endpoint vea el evento_id y el timestamp generados en el servidor.
    return evento
```

Con --reload activo, uvicorn recarga solo. Abre http://localhost:8000/docs y confirma que aparece POST /notificaciones, con el esquema NotificacionEvent como cuerpo esperado.

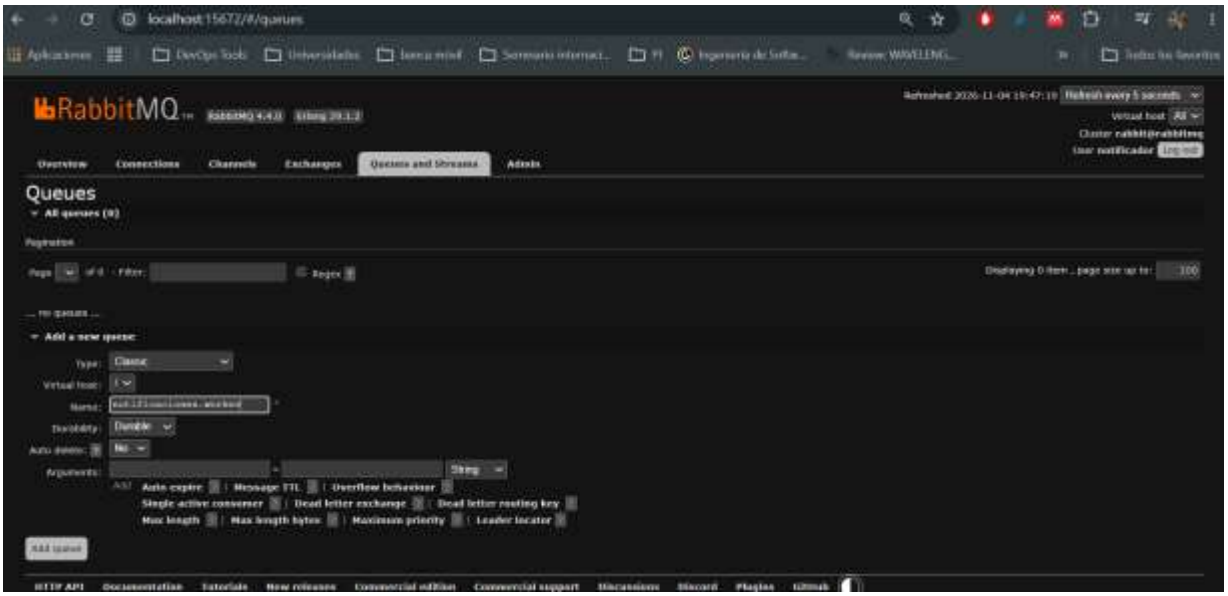


Paso 6: Probar el Productor (Swagger + consola de RabbitMQ)

En Kafka, el mensaje queda en el tópico aunque nadie lo lea. En RabbitMQ no: el exchange solo enruta, y si no hay una cola enlazada, el mensaje se descarta. Antes de publicar hay que crear una cola y enlazarla al exchange. Todavía no existe el worker, así que se hace a mano desde la consola, lo cual permite ver cada concepto del contexto arquitectónico en acción.

1. En http://localhost:15672, pestaña **Queues and Streams**, sección *Add a new queue*:

- Type: **Classic**
- Name: **notificaciones.worker**
- Durability: **Durable**



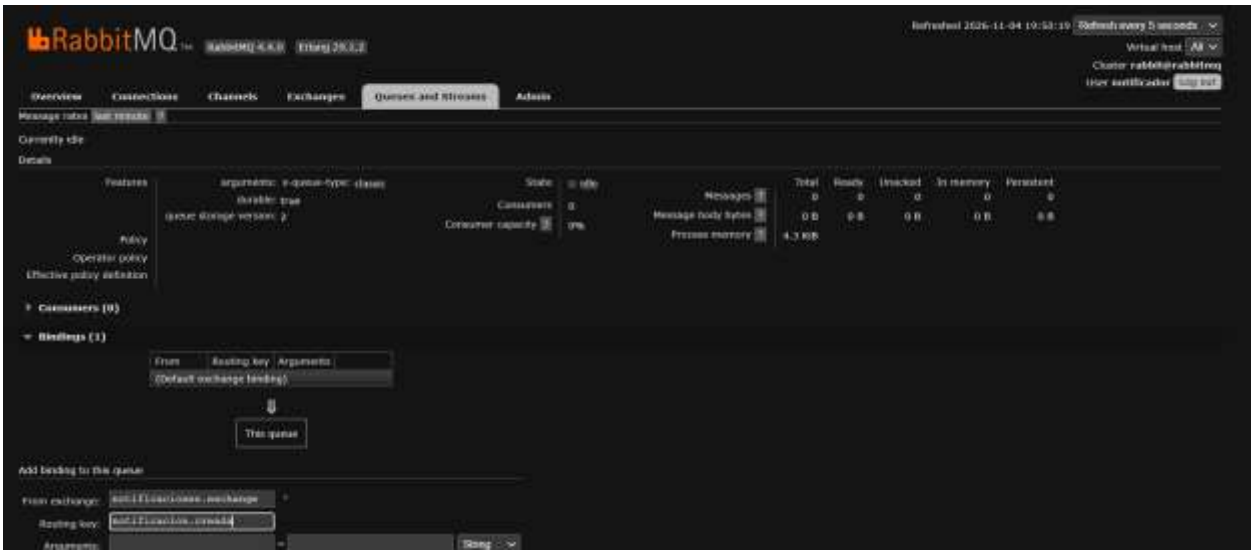
Clic en *Add queue*. Es importante que Type y Durability coincidan con lo indicado: más adelante el worker declarará esta misma cola con esos parámetros, y si difieren, RabbitMQ rechaza la declaración.



2. Haz clic sobre la cola recién creada, abre la sección **Bindings** y agrega:

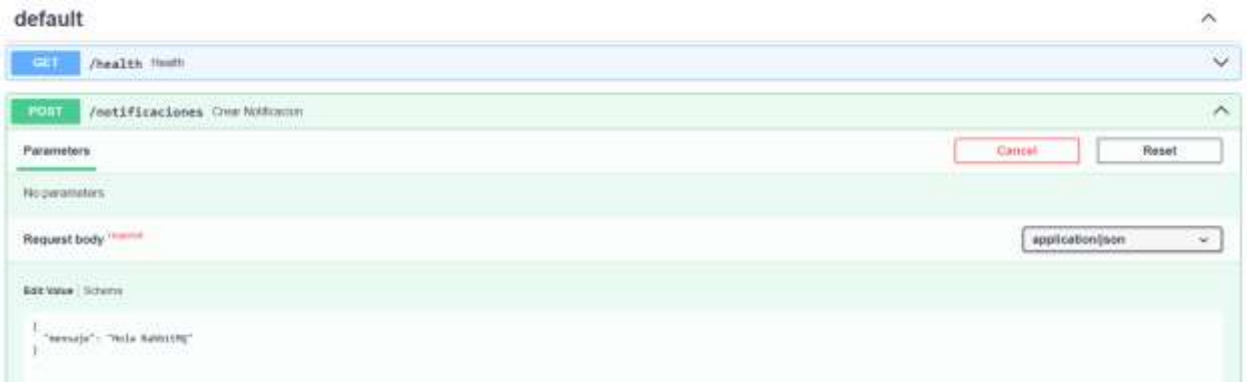
- From exchange: **notificaciones.exchange**
- Routing key: **notificacion.creada**

Clic en *Bind*. Con esto, todo mensaje publicado en el exchange con esa routing key llegará a la cola.



3. En http://localhost:8000/docs, despliega POST /notificaciones, clic en *Try it out* y reemplaza el body por:

```
{
  "mensaje": "Hola RabbitMQ"
}
```



Clic en *Execute*. Debe responder con código 200 y un JSON con evento_id, mensaje y timestamp completos. Eso confirma que el endpoint y la publicación funcionaron sin errores del lado de la API.



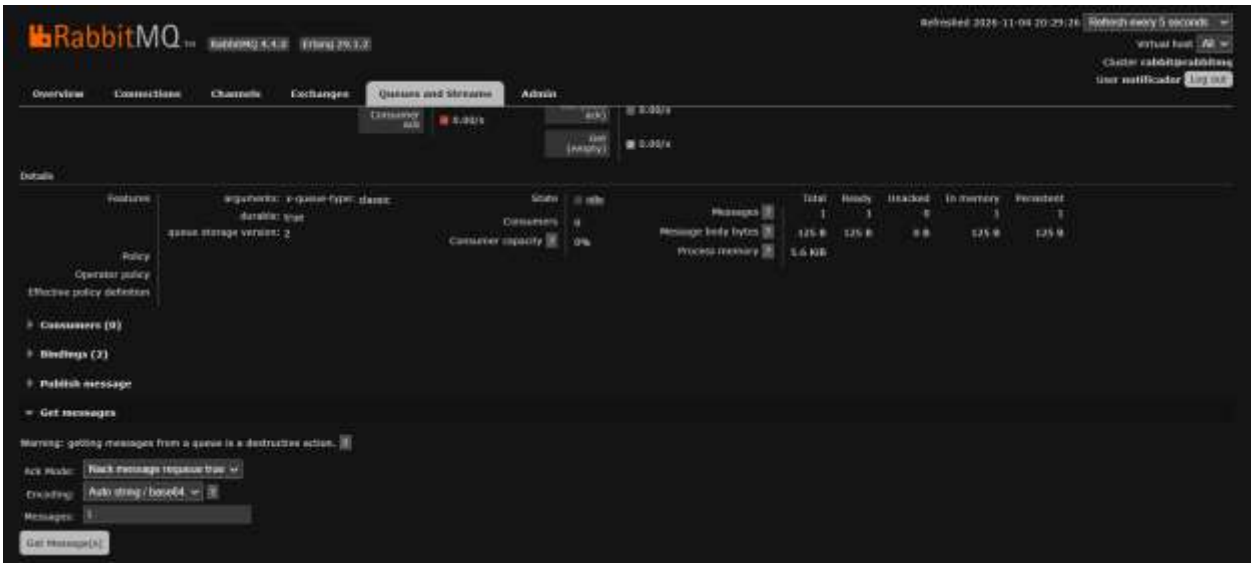
4. Confirma que el mensaje llegó a la cola, sin pasar por ningún código propio:

```
docker exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
```

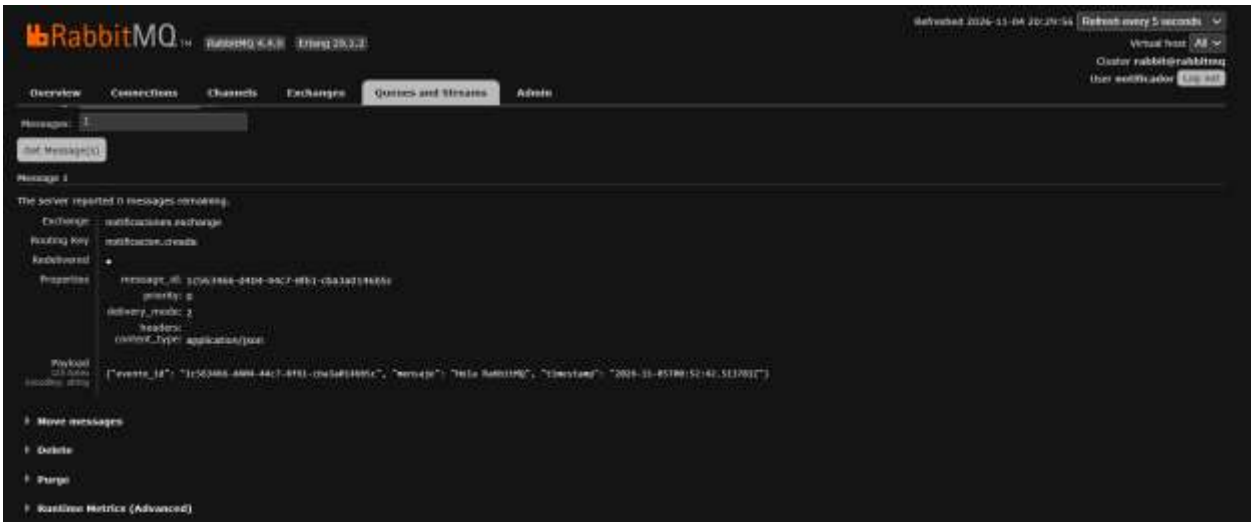
```
Manuel Pastrana@DESKTOP-7FO66S7 MINGW64 /f/Workspace/dojo-rabbitmq-notificaciones
$ docker exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificaciones.worker   1       0
```

Debe mostrar notificaciones.worker con messages_ready en 1 y messages_unacknowledged en 0: un mensaje esperando a que alguien lo tome.

5. Para ver el contenido del mensaje sin consumirlo, entra a la pestaña **Queues and Streams**, en el navegador, dentro de la consola de administración. Ahí, haz clic sobre la cola **notificaciones.worker** y despliega la sección **Get messages**. Deja *Ack Mode* en *Nack message requeue true*, deja *Messages* en 1 y haz clic en *Get Message(s)*. En el campo *Payload* debe aparecer el JSON publicado: {“evento_id”: “...”, “mensaje”: “Hola RabbitMQ”, “timestamp”: “...”}..



Con ese modo de lectura el mensaje se devuelve a la cola, de modo que sigue esperando al worker.



Experimento opcional. En la consola, elimina el binding de la cola (sección Bindings, *Unbind*) y publica otro mensaje desde Swagger. La API responde 200, pero la cola no recibe nada: el exchange descartó el mensaje. Vuelve a crear el binding antes de continuar.

Paso 7: Crear el Proyecto notificador-worker

Abre una nueva pestaña de terminal en VS Code y deja intacta la que tiene uvicorn corriendo, de modo que la API siga viva mientras se arma el worker.

1. Desde la raíz del proyecto, crea la carpeta del worker:

```
cd dojo-rabbitmq-notificaciones
mkdir notificador-worker
cd notificador-worker
```

2. Crea y activa su propio entorno virtual, independiente del de notificador-api. Cada servicio tiene el suyo, y así queda claro que son proyectos separados:

```
python -m venv venv
source venv/Scripts/activate
```

3. Con notificador-worker seleccionado, crea requirements.txt:

```
aio-pika
pydantic
```

Aquí no van fastapi ni uvicorn. El worker no expone una API HTTP: solo consume de RabbitMQ y persiste, por lo que no necesita un servidor web.

4. Instala las dependencias:

```
pip install -r requirements.txt
```

5. Crea .gitignore en la carpeta notificador-worker, al mismo nivel que requirements.txt (no dentro de venv/). Este archivo le indica a git qué no debe versionar ni el entorno virtual, ni el caché de Python (__pycache__/) y tampoco el archivo .env, donde suelen guardarse credenciales.

```
venv/
__pycache__/
.env
data/
```

Paso 8: Implementar el Consumidor con ack explícito

1. Con notificador-worker seleccionada, crea schemas.py con el mismo contrato que la API (se copia, no se importa, porque son proyectos independientes):

```
from datetime import datetime, timezone
from uuid import uuid4

from pydantic import BaseModel, Field



# Esta clase es una copia de la que está en notificador-api/schemas.py,
# no una importación. Es intencional: en el mundo real estos dos
# servicios viven en repositorios y despliegues separados. Lo que los
# mantiene compatibles es que ambos respeten los mismos nombres de campo
# y tipos, no que reutilicen una clase en común.
class NotificacionEvent(BaseModel):
    evento_id: str = Field(default_factory=lambda: str(uuid4()))
    mensaje: str
    timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))
```

2. Crea consumer.py al mismo nivel de requirements.txt y schemas.py. Copia y pega el siguiente contenido para analizarlo:

```
import os

import aio_pika
from aio_pika import ExchangeType
from aio_pika.abc import AbstractIncomingMessage

from schemas import NotificacionEvent

# Misma lógica que en el productor: variable de entorno con valor por
# defecto para desarrollo local.
RABBITMQ_URL = os.environ.get(
    'RABBITMQ_URL', 'amqp://notificador:notificador123@localhost:5672/'
)

# Deben coincidir exactamente con lo que declara notificador-api.
EXCHANGE_NAME = 'notificaciones.exchange'
ROUTING_KEY = 'notificacion.creada'

# El nombre de la cola cumple aquí el papel que tenía el group_id en
# Kafka: es lo que identifica a este consumidor ante el broker. Si se
# levantara una segunda instancia de este worker con la MISMA cola,
# RabbitMQ repartiría los mensajes entre ambas en vez de entregarlos dos
# veces.
QUEUE_NAME = 'notificaciones.worker'


async def procesar(mensaje: AbstractIncomingMessage) -> None:
    try:
        # Reconstruir el evento contra el esquema da validación: si llegara
        # un mensaje que no cumple el contrato, falla aquí con un error
        # claro. ValidationError es subclase de ValueError.
        evento = NotificacionEvent.model_validate_json(mensaje.body)
    except ValueError:
        # Un mensaje inválido no mejora si se reintenta: se rechaza sin
        # devolverlo a la cola (requeue=False).
        await mensaje.reject(requeue=False)
        return

    print(f'Notificación recibida: {evento}')

    # El ack le dice a RabbitMQ que el mensaje ya fue procesado y puede
    # borrarlo de la cola. Si el worker se cae antes de este punto, el
    # mensaje sigue en la cola y se reentrega.
    await mensaje.ack()


async def consumir_notificaciones() -> None:
    connection = await aio_pika.connect_robust(RABBITMQ_URL)
    try:
        channel = await connection.channel()
        # prefetch_count=1: RabbitMQ no entrega un mensaje nuevo hasta que
        # el anterior fue confirmado. Sin este límite, el broker enviaría
        # al worker todos los mensajes de golpe.
        await channel.set_qos(prefetch_count=1)
```

```
        # Declarar es idempotente, siempre que los parámetros coincidan con
        # los de la cola ya existente (la creada a mano en el Paso 6).
        exchange = await channel.declare_exchange(
            EXCHANGE_NAME, ExchangeType.DIRECT, durable=True
        )
        queue = await channel.declare_queue(QUEUE_NAME, durable=True)
        await queue.bind(exchange, routing_key=ROUTING_KEY)

        # 'async for' mantiene este bucle corriendo indefinidamente,
        # procesando un mensaje a la vez. Por defecto la entrega exige ack
        # manual.
        async with queue.iterator() as mensajes:
            async for mensaje in mensajes:
                await procesar(mensaje)
    finally:
        # Cierre ordenado de la conexión si el bucle se interrumpe (por
        # ejemplo, con Ctrl+C).
        await connection.close()
```

3. Crea main.py:

```
import asyncio

from consumer import consumir_notificaciones

# A diferencia de notificador-api (que usa uvicorn como punto de entrada),
# este worker no es un servidor web: es un proceso que corre un bucle
# infinito por su cuenta.
if __name__ == '__main__':
    try:
        asyncio.run(consumir_notificaciones())
    except KeyboardInterrupt:
        # Sin este bloque, Ctrl+C muestra un traceback completo de asyncio.
        # No es un error real; se captura para que la salida se vea
        # intencional.
        print('Worker detenido.')
```

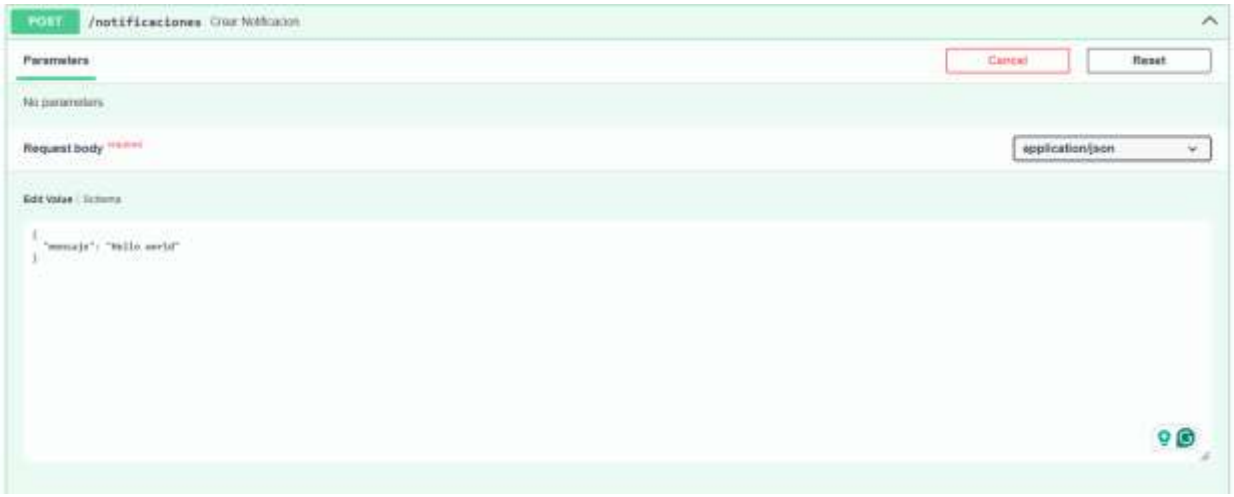
4. Ejecuta:

```
python main.py
```

Checkpoint importante: el mensaje "Hola RabbitMQ" publicado en el Paso 6 sigue esperando en la cola, así que debe imprimirse de inmediato. Después, en la consola (Queues and Streams), la cola debe mostrar Ready 0 y Unacked 0: al recibir el ack, RabbitMQ borró el mensaje.



Con el worker todavía corriendo, publica una notificación nueva desde Swagger (http://localhost:8000/docs). Sin reiniciar nada, debe aparecer impresa en la terminal del worker casi de inmediato. Esa es la prueba real del desacoplamiento: ningún proceso llamó al otro directamente.





Paso 9: Persistir lo Consumido (SQLite)

Hasta aquí el worker solo imprime lo que recibe; si el proceso se reiniciara, esa información se perdería. Un consumidor real persiste lo que procesa o dispara una acción (llamar a otro servicio, escribir en una base de datos); no se queda solo en memoria.

Para eso crearemos un archivo que permite el almacenamiento en SQLite y lo llamaremos storage.py. Esa clase va en notificador-worker/, al mismo nivel que main.py, consumer.py, schemas.py y requirements.txt. No va en notificador-api.

1. Crea storage.py:

```
import os
import sqlite3


# SQLite es parte de la librería estándar de Python: no agrega ninguna
# dependencia al proyecto y da un archivo de base de datos real y
# consultable.
#
# Los datos persistentes viven bajo data/, separados del código de la
# app. Esto permite montar después (Paso 11) un volumen de Docker solo
# sobre data/, sin tapar el código que se copia al construir la imagen.
DATA_DIR = 'data'
DB_PATH = os.path.join(DATA_DIR, 'notificaciones.db')



def init_db() -> None:
    # makedirs crea la carpeta data/ si no existe todavía (por ejemplo, la
    # primera vez que el contenedor arranca con un volumen vacío).
    os.makedirs(DATA_DIR, exist_ok=True)
```

```
    # CREATE TABLE IF NOT EXISTS permite llamar esta función en cada
    # arranque sin verificar antes si la tabla ya existe.
    with sqlite3.connect(DB_PATH) as conexion:
        conexion.execute(
            '''
            CREATE TABLE IF NOT EXISTS notificaciones (
                evento_id TEXT PRIMARY KEY,
                mensaje TEXT NOT NULL,
                timestamp TEXT NOT NULL
            )
            '''
        )


def guardar_notificacion(evento_id: str, mensaje: str, timestamp: str) -> None:
    # INSERT OR IGNORE (en vez de un INSERT normal) es importante: RabbitMQ
    # garantiza entrega 'al menos una vez' cuando se usa ack manual, no
    # 'exactamente una vez'. Si el worker se cae justo después de guardar
    # pero antes de enviar el ack, RabbitMQ reentrega el mismo mensaje al
    # reconectarse. Como evento_id es la clave primaria, el segundo intento
    # de insertar el mismo evento_id se ignora en silencio en vez de fallar
    # o duplicar la fila.
    with sqlite3.connect(DB_PATH) as conexion:
        conexion.execute(
            'INSERT OR IGNORE INTO notificaciones (evento_id, mensaje, timestamp) '
            'VALUES (?, ?, ?)',
            (evento_id, mensaje, timestamp),
        )
```

2. Reemplaza en la clase consumer.py para que incluya la persistencia:

```
import os
import aio_pika
from aio_pika import ExchangeType
from aio_pika.abc import AbstractIncomingMessage
from schemas import NotificacionEvent
from storage import guardar_notificacion

# Misma lógica que en el productor: variable de entorno con valor por
# defecto para desarrollo local.
RABBITMQ_URL = os.environ.get(
    'RABBITMQ_URL', 'amqp://notificador:notificador123@localhost:5672/'
)

# Deben coincidir exactamente con lo que declara notificador-api.
EXCHANGE_NAME = 'notificaciones.exchange'
ROUTING_KEY = 'notificacion.creada'

# El nombre de la cola cumple aquí el papel que tenía el group_id en
# Kafka: es lo que identifica a este consumidor ante el broker. Si se
# levantara una segunda instancia de este worker con la MISMA cola,
# RabbitMQ repartiría los mensajes entre ambas en vez de entregarlos dos
# veces.
QUEUE_NAME = 'notificaciones.worker'
```

```
async def procesar(mensaje: AbstractIncomingMessage) -> None:
    try:
        # Reconstruir el evento contra el esquema da validación: si llegara
        # un mensaje que no cumple el contrato, falla aquí con un error
        # claro. ValidationError es subclase de ValueError.
        evento = NotificacionEvent.model_validate_json(mensaje.body)
    except ValueError:
        # Un mensaje inválido no mejora si se reintenta: se rechaza sin
        # devolverlo a la cola (requeue=False).
        await mensaje.reject(requeue=False)
        return

    try:
        # timestamp.isoformat() convierte el datetime a texto, porque
        # SQLite no tiene un tipo datetime nativo.
        guardar_notificacion(
            evento.evento_id, evento.mensaje, evento.timestamp.isoformat()
        )
    except Exception:
        # Si falla la persistencia, el mensaje vuelve a la cola para no
        # perderse, y se deja subir el error para que el fallo sea visible.
        await mensaje.nack(requeue=True)
        raise

    # El ack le dice a RabbitMQ que el mensaje ya fue procesado y puede
    # borrarlo de la cola. Va DESPUÉS de persistir: si se confirmara antes
    # y el worker se cayera al guardar, el mensaje se perdería para siempre.
    # Si el worker se cae antes de este punto, el mensaje sigue en la cola
    # y se reentrega.
    await mensaje.ack()
    print(f'Notificación recibida y persistida: {evento}')


async def consumir_notificaciones() -> None:
    connection = await aio_pika.connect_robust(RABBITMQ_URL)
    try:
        channel = await connection.channel()
        # prefetch_count=1: RabbitMQ no entrega un mensaje nuevo hasta que
        # el anterior fue confirmado. Sin este límite, el broker enviaría
        # al worker todos los mensajes de golpe.
        await channel.set_qos(prefetch_count=1)

        # Declarar es idempotente, siempre que los parámetros coincidan con
        # los de la cola ya existente (la creada a mano en el Paso 6).
        exchange = await channel.declare_exchange(
            EXCHANGE_NAME, ExchangeType.DIRECT, durable=True
        )
        queue = await channel.declare_queue(QUEUE_NAME, durable=True)
        await queue.bind(exchange, routing_key=ROUTING_KEY)

        # 'async for' mantiene este bucle corriendo indefinidamente,
        # procesando un mensaje a la vez. Por defecto la entrega exige ack
        # manual.
        async with queue.iterator() as mensajes:
            async for mensaje in mensajes:
                await procesar(mensaje)
```

```
    finally:
        # Cierre ordenado de la conexión si el bucle se interrumpe (por
        # ejemplo, con Ctrl+C).
        await connection.close()
```

3. Agrega en main.py el import de la clase storage para consumir el init_db, e inicializar la BD al iniciar.:

```
import asyncio

from consumer import consumir_notificaciones
from storage import init_db

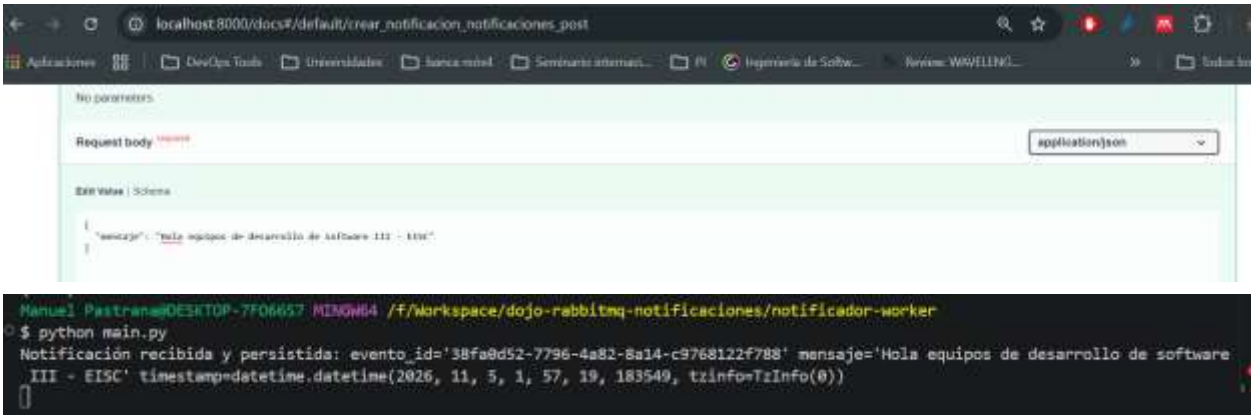
# A diferencia de notificador-api (que usa uvicorn como punto de entrada),
# este worker no es un servidor web: es un proceso que corre un bucle
# infinito por su cuenta.
# init_db() debe correr ANTES de que el consumidor empiece a recibir
# mensajes: si la tabla no existe todavía y llega un mensaje, el primer
# guardar_notificacion() fallaría.
if __name__ == '__main__':
    init_db()
    try:
        asyncio.run(consumir_notificaciones())
    except KeyboardInterrupt:
        print('Worker detenido.')
```

4. Detén el worker (Ctrl+C) y vuelve a levantarlo:

```
python main.py
```

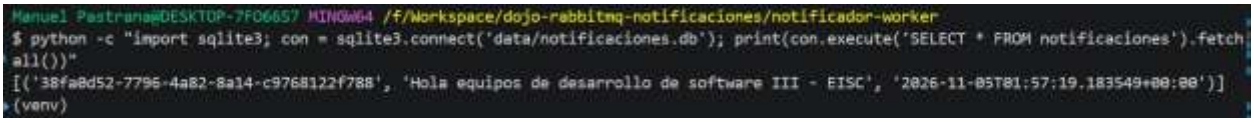
Como el ack de los mensajes anteriores ya se envió, el worker no los vuelve a procesar al reiniciar; solo persiste los mensajes nuevos. Es el comportamiento correcto: a diferencia de Kafka, en RabbitMQ un mensaje confirmado desaparece de la cola.

5. Publica una notificación nueva desde Swagger y confirma que en la terminal del worker aparece "Notificación recibida y persistida".



6. Verifica que quedó en SQLite (desde la carpeta notificador-worker, en una sola línea):

```
python -c "import sqlite3; con = sqlite3.connect('data/notificaciones.db');
print(con.execute('SELECT * FROM notificaciones').fetchall())"
```



Paso 10: Dockerizar Ambos Servicios

notificador-api

1. Crea Dockerfile en notificador-api, al mismo nivel del main.py:

```
# Imagen base ligera: 'slim' trae solo lo esencial para correr Python,
# sin las herramientas de compilación de la imagen completa.
FROM python:3.13-slim

# Todo lo que sigue se ejecuta dentro de esta carpeta del contenedor.
WORKDIR /app

# Se copia primero solo requirements.txt para aprovechar el caché de
# capas de Docker: si el código cambia pero las dependencias no, la capa
# de instalación no se vuelve a ejecutar.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Ahora sí se copia el resto del código.
COPY . .

# host 0.0.0.0 (no 127.0.0.1) es obligatorio: dentro de un contenedor,
# 'localhost' solo acepta conexiones del mismo contenedor. Para que Docker
# pueda exponer el puerto hacia afuera, uvicorn debe escuchar en todas las
# interfaces.
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

2. Crea .dockerignore en notificador-api:

```
# El entorno virtual se creó con el Python de Windows y no funcionaría
# dentro del contenedor Linux. El Dockerfile ya instala las dependencias
# de cero con pip install.
venv/
__pycache__/
.git/
.gitignore
```

3. Construye la imagen:

```
docker build -t notificador-api .
```



notificador-worker

1. Crea Dockerfile en notificador-worker:

```
FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Este servicio no expone ningún puerto (no es un servidor web), así que
# el CMD simplemente ejecuta el script que arranca el bucle del consumidor.
CMD ["python", "main.py"]
```

2. Crea .dockerignore en notificador-worker:

```
venv/
__pycache__/
.git/
.gitignore
# La base de datos de desarrollo no debe quedar dentro de la imagen: el
# contenedor crea la suya al arrancar.
data/
```

3. Construye la imagen:

```
docker build -t notificador-worker .
```



Los contenedores todavía no se pueden ejecutar de forma independiente: necesitan estar en la misma red que RabbitMQ. Eso es lo que resuelve el docker-compose.yml final del siguiente paso.

Paso 11: docker-compose.yml Completo (RabbitMQ + API + Worker)

Deteniendo los procesos sueltos

Antes de levantar la versión final, hay que detener todo lo que sigue corriendo fuera de Docker, en este orden: Ctrl+C en la terminal del worker, Ctrl+C en la terminal de uvicorn (si no, el puerto 8000 quedaría ocupado) y, desde la raíz del proyecto:

```
docker-compose down
```

docker-compose.yml final, en la raíz del proyecto

El servicio rabbitmq queda igual que en el Paso 1. Se agregan la API y el worker, y el volumen del worker.

```
# Imagen oficial de RabbitMQ con el plugin de administración web.
services:
  rabbitmq:
    image: rabbitmq:4-management
    container_name: rabbitmq
    # Nombre de host fijo. RabbitMQ identifica su nodo como
    # "rabbit@<hostname>" y guarda sus datos bajo ese nombre. Con el
    # hostname por defecto de Docker (el id del contenedor), cada
    # contenedor nuevo sería un nodo distinto y no vería las colas
    # guardadas en el volumen.
    hostname: rabbitmq
    ports:
      # 5672: protocolo AMQP, el que usan los clientes (API y worker).
      - '5672:5672'
      # 15672: consola de administración web.
      - '15672:15672'
    environment:
      # Usuario y clave propios para el laboratorio, en lugar del
      # usuario guest/guest que trae RabbitMQ por defecto (restringido a
      # conexiones locales en una instalación estándar). Solo se aplican
      # la primera vez que se crea el volumen de datos.
      RABBITMQ_DEFAULT_USER: notificador
      RABBITMQ_DEFAULT_PASS: notificador123
    volumes:
      # Las colas y los mensajes persistentes sobreviven a un
      # docker-compose down/up.
      - rabbitmq_data:/var/lib/rabbitmq
    # Evita que otros servicios intenten conectarse antes de que
    # RabbitMQ esté listo para aceptar conexiones.
    # check_port_connectivity y no 'ping': ping solo comprueba que el nodo
    # está vivo, y puede dar sano antes de que el puerto 5672 acepte
```

```
    # conexiones, lo que haría caer a la API y al worker al arrancar.

    healthcheck:
      test: ['CMD', 'rabbitmq-diagnostics', '-q', 'check_port_connectivity']

      interval: 10s
      timeout: 10s
      retries: 5
      start_period: 20s

  notificador-api:
    build: ./notificador-api
    container_name: notificador-api
    ports:
      - '8000:8000'
    environment:
      # Sobrescribe el valor por defecto ('localhost') definido en
      # producer.py. No se toca el código, solo la variable de entorno
      # para este contexto. 'rabbitmq' es el nombre del servicio en la red
      # que Docker Compose crea automáticamente.
      RABBITMQ_URL: amqp://notificador:notificador123@rabbitmq:5672/
    depends_on:
      # Sin esto, la API podría arrancar antes de que RabbitMQ acepte
      # conexiones y fallar en start_producer().
      rabbitmq:
        condition: service_healthy

  notificador-worker:
    build: ./notificador-worker
    container_name: notificador-worker
    environment:
      RABBITMQ_URL: amqp://notificador:notificador123@rabbitmq:5672/
    depends_on:
      rabbitmq:
        condition: service_healthy
    # Monta el volumen solo sobre data/, no sobre todo /app, para que el
    # SQLite sobreviva a un docker-compose down/up sin tapar el código de
    # la aplicación que se copió al construir la imagen.
    volumes:
      - worker_data:/app/data

volumes:
  rabbitmq_data:
  worker_data:
```

A diferencia de la guía de Kafka, aquí no hay listeners ni direcciones anunciadas que ajustar: un cliente AMQP se conecta a la dirección que se le da y no recibe del broker otra dirección a la cual reconectarse. El único cambio entre ejecutar en el equipo y ejecutar en Docker es el host de RABBITMQ_URL: localhost fuera de Docker, rabbitmq dentro.

1. Levanta todo:

```
docker-compose up -d --build
docker-compose ps -a
```

> - **Nota. -d** (detached) levanta los contenedores en segundo plano y te devuelve la terminal. Sin él, docker-compose up se queda corriendo en primer plano mostrando los logs de los tres servicios, y al cerrar la terminal o presionar Ctrl+C los contenedores se detienen. Con -d siguen corriendo, y los logs se consultan cuando quieras con docker logs notificador-worker.
> - **--build** obliga a Docker a construir de nuevo las imágenes de notificador-api y notificador-worker a partir de sus Dockerfile antes de arrancar. Sin ese flag, si ya existe una imagen construida antes, Compose la reutiliza, aunque hayas cambiado el código, y verías el comportamiento viejo. Con --build, cada vez que modificas main.py, consumer.py u otro archivo, la imagen se reconstruye con esos cambios. Para rabbitmq no tiene efecto, porque usa una imagen ya publicada (image:), no una que se construya aquí.

Los tres contenedores (rabbitmq, notificador-api, notificador-worker) deben aparecer como Up, y rabbitmq además como healthy.



Paso 12: Prueba End-to-End desde cero

Para confirmar que la guía funciona igual de bien para quien la sigue desde cero, sin el historial de ajustes hecho hasta aquí, conviene repetir el ciclo completo de arranque limpio al menos una vez.

1. Apaga todo y borra también los volúmenes, para garantizar un estado completamente limpio (sin colas ni datos previos):

```
docker-compose down -v
```

2. Levanta todo de nuevo:

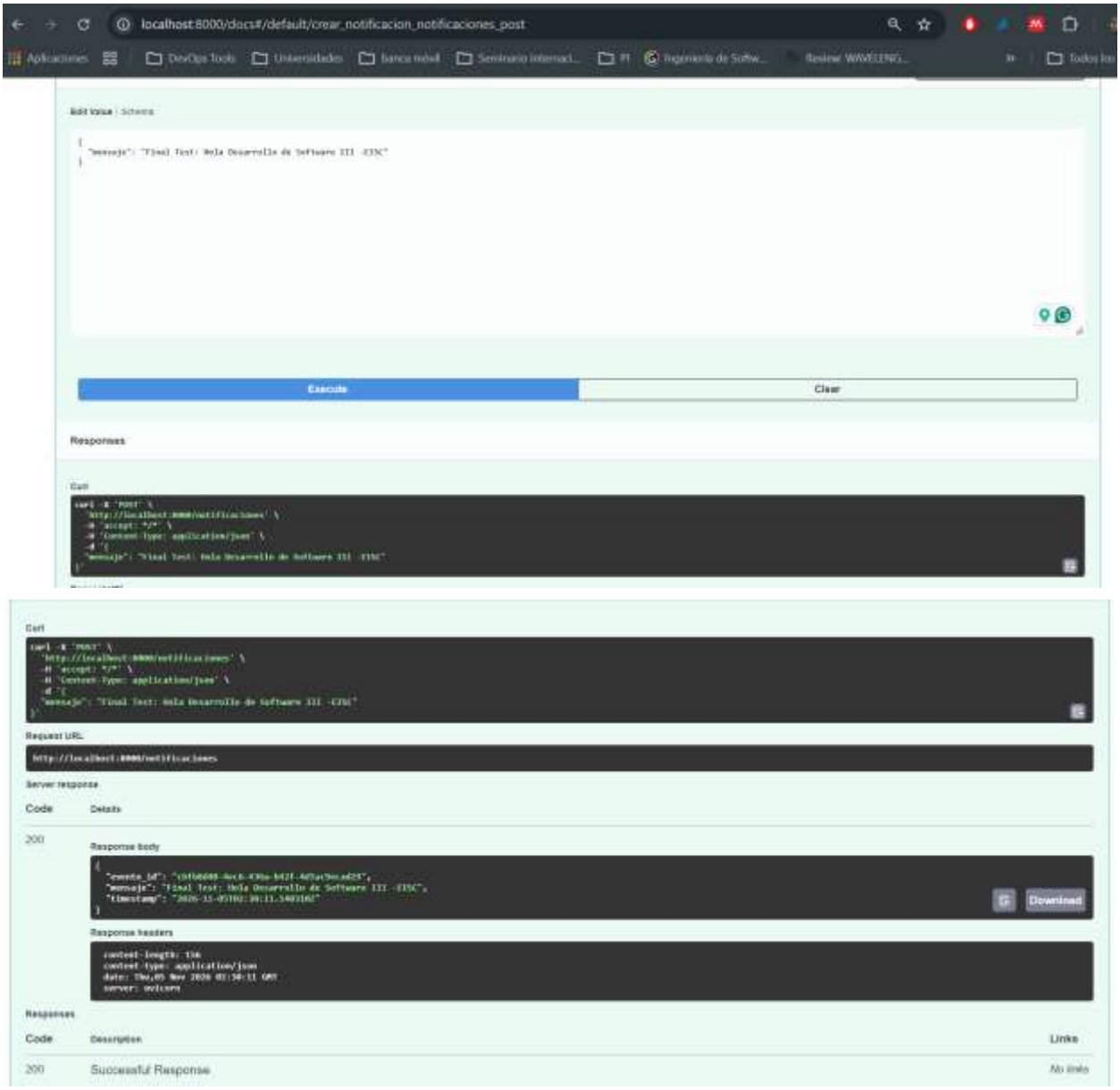
```
docker-compose up -d --build
```

3. Espera de 20 a 30 segundos (RabbitMQ necesita ese tiempo para estar healthy y para que el worker declare su cola) y confirma que los tres contenedores están arriba:

```
docker ps -a
```



4. Publica dos o tres notificaciones distintas desde http://localhost:8000/docs.



5. Confirma que la cola quedó vacía, es decir, que todos los mensajes fueron confirmados:

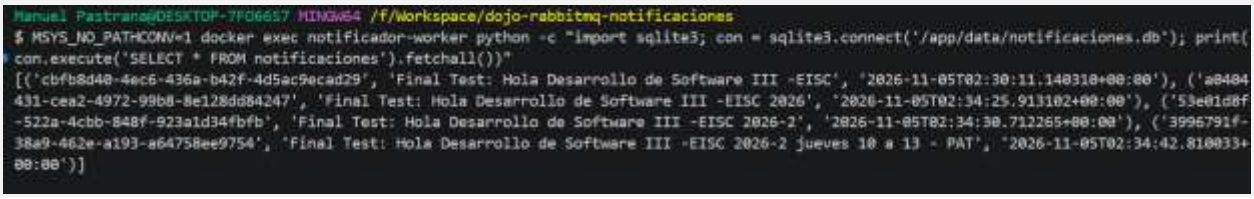
```
docker exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
```

```
Manuel Pastrana@DESKTOP-7FO66S7 MINGW64 /f/Workspace/dojo-rabbitmq-notificaciones
$ docker exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificaciones.worker   0       0
(venv)
```

notificaciones.worker debe mostrar 0 en messages_ready y 0 en messages_unacknowledged.

6. Confirma que todas quedaron en SQLite (en una sola línea):

```
MSYS_NO_PATHCONV=1 docker exec notificador-worker python -c "import sqlite3; con =
sqlite3.connect('/app/data/notificaciones.db'); print(con.execute('SELECT * FROM
notificaciones').fetchall())"
```



7. Prueba la durabilidad: detén solo el worker, publica dos notificaciones más y revisa la cola. Deben quedar acumuladas en messages_ready; al volver a levantar el worker, se procesan solas.

```
docker-compose stop notificador-worker
docker exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
docker-compose start notificador-worker
```

> **Orden de arranque tras un down -v.** Con los volúmenes recién borrados, la cola no existe hasta que el worker la declara. Si se publica antes de que el worker termine de arrancar, esos mensajes se descartan. Por eso se espera de 20 a 30 segundos antes del paso 4.

Solución de problemas comunes

Referencia rápida de los problemas más probables al seguir esta guía.

Comandos partidos en varias líneas

Al copiar un comando largo desde un PDF o desde un editor con ajuste de línea, el salto de línea viaja con el texto y la terminal lo interpreta como dos comandos distintos. El síntoma típico es un error de argumentos incompletos seguido de un command not found. Pega los comandos en una sola línea.

Git Bash en Windows convierte rutas absolutas de Linux

MINGW64 traduce automáticamente cualquier argumento que empiece con / a una ruta de Windows antes de pasarlo a Docker, lo que rompe los comandos que usan rutas absolutas de Linux, como /app/data/notificaciones.db. La solución es anteponer MSYS_NO_PATHCONV=1 al comando. No aplica a WSL, Linux ni Mac, ni a los comandos de esta guía que no contienen rutas absolutas.

localhost contra rabbitmq en RABBITMQ_URL

El host de RABBITMQ_URL depende de dónde corre el proceso. Si corre en el equipo (Pasos 1 a 9), debe ser localhost. Si corre en un contenedor de la misma red de Compose (Paso 11 en adelante), debe ser rabbitmq. Usar rabbitmq desde el equipo produce un error de resolución de nombre (getaddrinfo failed); usar localhost dentro de un contenedor produce Connection refused, porque localhost es el propio contenedor.

ACCESS_REFUSED al conectar

El usuario o la clave de la URL no coinciden con los que RabbitMQ tiene guardados. RABBITMQ_DEFAULT_USER y RABBITMQ_DEFAULT_PASS solo se aplican cuando se crea el volumen de datos. Si se editaron en el compose después de la primera ejecución, el cambio no surte efecto hasta ejecutar docker-compose down -v.

PRECONDITION_FAILED: inequivalent arg

Una cola o un exchange ya existe con parámetros distintos a los que intenta declarar el código (por ejemplo, la cola creada en la consola como Quorum y declarada en el worker sin argumentos). RabbitMQ no modifica elementos existentes: o se alinean los parámetros, o se elimina la cola en la consola (pestaña Queues and Streams, opción Delete) o con docker-compose down -v, y se deja que el worker la cree.

Publiqué un mensaje, la API respondió 200, pero no llegó a ninguna cola

El exchange no tiene ninguna cola enlazada para esa routing key y descartó el mensaje en silencio. Revisa en la consola, dentro de la cola, que el binding exista y que el exchange y la routing key sean exactamente notificaciones.exchange y notificacion.creada (un error de una letra produce este mismo síntoma).

Mensajes en estado Unacked que no se mueven

El worker recibió el mensaje pero no envió el ack. Con prefetch_count=1, mientras haya un mensaje sin confirmar, RabbitMQ no entrega más: la cola parece detenida. Revisa que mensaje.ack() se ejecute en todos los caminos del código. Si el worker se detiene, los mensajes sin confirmar vuelven a estar en Ready.

Las colas desaparecen después de recrear el contenedor

Si RabbitMQ pierde sus datos tras un down/up sin -v, revisa que el servicio tenga hostname: rabbitmq fijo y el volumen montado en /var/lib/rabbitmq. Sin hostname fijo, cada contenedor nuevo es un nodo distinto y no encuentra los datos del volumen anterior.

Puertos 5672 o 15672 ocupados

Otro proceso (por ejemplo, una instalación local de RabbitMQ) ya usa el puerto. Detenlo o cambia el lado izquierdo del mapeo en el compose (por ejemplo, '5673:5672').

VS Code crea archivos en la carpeta seleccionada

Al usar clic derecho > New File, el archivo se crea dentro de la carpeta seleccionada en el explorador, no necesariamente en la raíz del servicio. Es fácil terminar creando requirements.txt o .gitignore dentro de venv/ por accidente. Verifica siempre el breadcrumb sobre el editor antes de pegar contenido.

KeyboardInterrupt al detener el worker con Ctrl+C

Sin un bloque try/except alrededor de asyncio.run(), presionar Ctrl+C en notificador-worker imprime un traceback completo de asyncio. No es un error real: el bloque finally que cierra la

conexión con RabbitMQ se ejecuta antes de que la excepción termine de propagarse. El try/except KeyboardInterrupt de main.py (Paso 8) existe solo para que la salida se vea intencional.

Actividad de reflexión

1. ¿Por qué notificador-api y notificador-worker nunca se llaman directamente entre sí? ¿Qué ventaja práctica trae ese desacoplamiento en un sistema real, más allá de este laboratorio?
2. Si se levantara una segunda instancia de notificador-worker que consume de la misma cola (notificaciones.worker), ¿qué pasaría con los mensajes? ¿Y si la segunda instancia declarara y consumiera de una cola distinta, enlazada al mismo exchange con la misma routing key?
3. ¿Por qué se usa INSERT OR IGNORE en vez de un INSERT normal al persistir las notificaciones? ¿Qué escenario real está previniendo esa decisión?
4. Si el worker se cayera justo después de recibir un mensaje pero antes de enviar el ack, ¿qué pasaría al reiniciarlo? ¿Qué tres decisiones del laboratorio (ack manual, cola durable, mensajes persistentes) hacen posible esa recuperación, y qué se perdería si faltara cada una?
5. Después de que un worker procesa y confirma un mensaje, ¿qué ocurre con él en RabbitMQ? ¿En qué se diferencia esto de lo que ocurre con un evento en Kafka, y qué consecuencias tiene para elegir una herramienta u otra?
6. Este laboratorio corre completamente en local. ¿Qué tendría que cambiar, no solo en código sino también en la configuración, para desplegarlo en la nube?

Reflexionar profundamente antes de ver las respuestas.

Posibles respuestas

1. ¿Por qué notificador-api y notificador-worker nunca se llaman directamente entre sí?

Porque ambos solo conocen nombres del broker (el exchange notificaciones.exchange, la routing key notificacion.creada y la cola notificaciones.worker), no la dirección, el puerto ni el protocolo del otro servicio. La API se encarga de publicar; el worker se encarga de consumir sin saber qué servicio publicó. Cada uno tiene su propia responsabilidad sin depender de lo que el otro haga.

¿Qué ventaja práctica trae ese desacoplamiento en un sistema real?

Si el worker se cae o está lento, la API sigue respondiendo con normalidad: los mensajes se acumulan en la cola hasta que el worker vuelva. Cada servicio puede desplegarse y escalar de forma independiente, con su propio ciclo de release y su propio equipo. Se puede agregar mañana un segundo consumidor (por ejemplo, auditoría) enlazando otra cola al mismo exchange, sin tocar una sola línea de la API. Y la cola actúa como amortiguador ante picos de carga, evitando que la API se sature esperando a que cada notificación termine de procesarse.

2. Segunda instancia con la misma cola.

RabbitMQ reparte los mensajes entre las dos instancias de forma alternada (patrón de consumidores competidores): cada mensaje lo recibe una sola instancia. Con prefetch_count=1, el reparto sigue la velocidad de cada una, y la instancia más rápida atiende más mensajes. Es la forma estándar de escalar un consumidor en RabbitMQ: más instancias sobre la misma cola. A diferencia de Kafka, el paralelismo no depende del número de particiones.

¿Y con una cola distinta enlazada al mismo exchange?

El exchange entrega una copia de cada mensaje a cada cola enlazada, así que ambos consumidores reciben todos los mensajes. Es el comportamiento adecuado cuando dos procesos distintos necesitan ver el flujo completo (por ejemplo, el worker que persiste en SQLite y otro que envía correos). Es el equivalente de dos grupos de consumidores con group_id distinto en Kafka.

3. ¿Por qué INSERT OR IGNORE?

Porque con ack manual RabbitMQ garantiza entrega "al menos una vez", no "exactamente una vez". Si el worker guarda el mensaje pero se cae antes de enviar el ack, RabbitMQ lo reentrega al reconectarse. Con un INSERT normal, ese segundo intento fallaría por violación de la clave primaria (evento_id) y el worker se caería intentando reprocesar algo que ya había guardado. INSERT OR IGNORE convierte ese escenario en un no-evento: si el evento_id ya existe, la inserción se ignora sin error ni duplicado. Es la forma más simple de hacer idempotente[1] al worker.

4. Caída después de recibir, pero antes del ack.

Para RabbitMQ, un mensaje entregado sin ack sigue siendo suyo. Al cerrarse la conexión del worker, el broker lo devuelve a la cola marcado como reentregado, y el worker, al reiniciar, lo recibe de nuevo. Eso funciona gracias a tres decisiones:

- **Ack manual y después de persistir.** Con ack automático, el mensaje se borra al entregarlo y una caída a mitad del procesamiento lo pierde. Con el ack antes de guardar, ocurre lo mismo.
- **Cola durable.** Si la cola no fuera durable, desaparecería al reiniciar el broker, junto con los mensajes pendientes.
- **Mensajes persistentes (delivery_mode PERSISTENT).** Si no lo fueran, el broker los guardaría solo en memoria y un reinicio del broker los perdería, aunque la cola siguiera existiendo.

El nombre de la cola cumple aquí el papel que el group_id cumple en Kafka: es la identidad a la que el broker asocia el estado pendiente.

5. ¿Qué ocurre con el mensaje después del ack y en qué se diferencia de Kafka?

En RabbitMQ, el mensaje se elimina de la cola cuando recibe el ack: la cola es un buffer de trabajo pendiente, no un historial. En Kafka, el evento permanece en el tópico según la política de retención, y cada grupo de consumidores solo avanza su propio offset; por eso un grupo nuevo puede leer todo desde el principio. Consecuencias: RabbitMQ encaja bien para tareas y comandos que deben procesarse una vez (envío de correos, trabajos en segundo plano) y ofrece enrutamiento flexible con exchanges. Kafka encaja mejor cuando se necesita conservar el

[1] Describe una operación que produce siempre el mismo resultado final sin importar si se ejecuta una vez o muchas veces con los mismos datos de entrada.

historial, reprocesar eventos pasados o alimentar muchos consumidores independientes con alto volumen.

6. ¿Qué tendría que cambiar para desplegarlo en la nube?

En código, casi nada: RABBITMQ_URL ya está externalizada como variable de entorno, y bastaría con apuntarla a un servicio administrado (por ejemplo, Amazon MQ para RabbitMQ o CloudAMQP). Donde sí cambia mucho es en la configuración y la infraestructura:

- **Seguridad de red:** pasar de amqp a amqps (TLS) con credenciales reales, y quitar el usuario de laboratorio.
- **Tamaño y alta disponibilidad:** pasar de un único nodo a un cluster de varios nodos, con colas replicadas (quorum queues) en lugar de colas clásicas.
- **Persistencia:** reemplazar SQLite por una base de datos administrada que soporte varias instancias del worker escribiendo a la vez.
- **Orquestación:** mover de docker-compose (un solo host) a Kubernetes, ECS o equivalente, con autoescalado y health checks de la plataforma.
- **Observabilidad:** logging centralizado, métricas de profundidad de cola y de mensajes sin confirmar, y alertas, en lugar de leer la salida de la terminal.
- **Mensajes fallidos:** una cola de mensajes muertos (dead-letter queue) para los mensajes rechazados, en lugar de descartarlos.
- **Manejo de secretos:** sacar las credenciales del docker-compose.yml en texto plano hacia un gestor de secretos real.